DIJKSTRA'S ALGORITHM

1. Definition

Dijkstra's Algorithm is a Greedy shortest path algorithm used to find the minimum distance from a source node to all other nodes in a weighted graph with non-negative edge weights.

2. Why Dijkstra? (Intuition)

- You start at a source node.
- Expand to the nearest unvisited node.
- Keep updating distances if a shorter route is found.
- Once a node's shortest distance is fixed, it **never changes**.

3. When Dijkstra Works?

✔ Works on

- Directed graphs
- Undirected graphs
- Weighted graphs (non-negative only

❌ Does NOT work on

- Negative edge weights
- Negative cycles

4. Why Dijkstra Fails for Negative Edge Weights?

Because Dijkstra assumes:

> Once a node's shortest distance is selected, it will never improve.

Negative edges can later reduce the distance → **breaks this assumption**. Bellman-Ford handles this case.

5. Key Terms

Term	Meaning
Visited Set	Nodes whose shortest distance is confirmed
Distance Array	Stores shortest known distance from source
Relaxation	Updating distance using `d[u] + w < d[v]`
Greedy Choice	Pick node with minimum tentative distance

6. Algorithm

```
// Algo Dijkstra(G, source)
// Input: Graph G with non-negative edge weights, source vertex
// Output: Shortest distance from source to all vertices

1. Initialize dist[] = ∞ for all vertices
2. dist[source] = 0
3. Mark all nodes as unvisited
4. Repeat until all nodes are visited:
       a. Select unvisited node u with minimum dist[u]
       b. Mark u as visited
       c. For each neighbor v of u:
              if dist[u] + weight(u,v) < dist[v]:
                    dist[v] = dist[u] + weight(u,v)
5. Return dist[]
```

7. Time Complexity

Implementation	Complexity
Simple array	**$O(V^2)$**
Min-heap (PQ)	**$O((V + E) \log V)$**
Fibonacci heap	**$O(E + V \log V)$**

10. Applications

- GPS navigation systems
- Network routing (OSPF uses Dijkstra)
- Road/Map optimization
- Game AI pathfinding
- Robot motion planning
- Packet routing in the Internet

11. Advantages

- Fast for non-negative weights
- Optimal shortest paths
- Works for large graphs using priority queue
- Simple and intuitive

12. Limitations

- Fails for negative weights
- Cannot detect negative cycles
- More complex than BFS
- Requires a priority queue for efficiency